INDEX

Sl No.	Experiments	Page No.
01	Implement Caesar Cipher substitution method	01-03
02	Implement Mono alphabetic cipher with the given key and input	04-05
03	Implement Poly alphabetic Cipher	06-07
04	Encrypt the given text using Play fair Cipher with the given key. Also perform the decryption operation	08-10
05	Implement Encryption and Decryption techniques in Hill Cipher	11-12
06	Demonstrate Single and Double Transposition techniques	13-14
07	Implement Simple DES/DES Algorithm	15-17
08	Generate Pseudo random numbers using Linear Congruential method	18-21
09	Generate Pseudo random numbers using Blum Blum Shub Generator	22-24
10	Implement RSA Algorithm	25-26
11	Demonstrate Diffie Hellman Key exchange Algorithm	27-28
12	Implement Fermat's and Euler's Theorem	29-34

1. Implement Caesar Cipher substitution method

```
import java.util.Scanner;

public class CaesarCipher {

   // Method to encrypt or decrypt a message

   public static String cipher(String text, int shiftKey) {

      // Use StringBuilder for efficient string manipulation

      StringBuilder result = new StringBuilder();

      for (char character : text.toCharArray()) {

         if (Character.isLetter(character)) {

            char base;

            // Determine the base character based on case ('A' for uppercase, 'a' for
lowercase)

            if (Character.isUpperCase(character)) {

               base = 'A';

            } else {

               base = 'a';

            }

            // Calculate the original position (0-25)

            int originalPosition = character - base;

            // Apply the shift using modular arithmetic to handle wrapping

            // The (shiftKey % 26 + 26) % 26 handles both positive and negative shifts
correctly

            int newPosition = (originalPosition + shiftKey % 26 + 26) % 26;

            // Convert back to a character and append to the result

            char newCharacter = (char) (base + newPosition);
```

```
            result.append(newCharacter);
        } else {
            // Keep non-alphabetic characters unchanged
            result.append(character);
        }
    }
    return result.toString();
  }
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter a message (plaintext):");
    String text = scanner.nextLine();
    System.out.println("Enter a shift key (integer):");
    int key = scanner.nextInt();
    // Encrypt the message
    String encryptedMessage = cipher(text, key);
    System.out.println("\nEncrypted message: " + encryptedMessage);
    // Decrypt the message (by shifting in the reverse direction using a negative key)
    String decryptedMessage = cipher(encryptedMessage, -key);
    System.out.println("Decrypted message: " + decryptedMessage);
    scanner.close();
  }
}
```

OUTPUT:

```
Enter a message (plaintext):
BKIT BHALKI
Enter a shift key (integer):
3

Encrypted message: ENLW EKDONL
Decrypted message: BKIT BHALKI
```

2. Implement Mono alphabetic cipher with the given key and input

```
import java.util.*;
public class MonoalphabeticCipher {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      // Define the standard alphabet
      String alphabet = "abcdefghijklmnopqrstuvwxyz";
      // Example Key: A full 26-character scrambled alphabet
      String key = "zyxwvutsrqponmlkjihgfedcba";
      System.out.print("Enter plaintext: ");
      String plaintext = sc.nextLine().toLowerCase();
      // 1. Create mapping for Encryption
      Map<Character, Character> encryptMap = new HashMap<>();
      for (int i = 0; i < alphabet.length(); i++) {
         encryptMap.put(alphabet.charAt(i), key.charAt(i));
      }
      // 2. Perform Encryption
      StringBuilder ciphertext = new StringBuilder();
      for (char c : plaintext.toCharArray()) {
         ciphertext.append(encryptMap.getOrDefault(c, c));
      }
      System.out.println("Ciphertext: " + ciphertext);
      // 3. Perform Decryption (Reverse Mapping)
      Map<Character, Character> decryptMap = new HashMap<>();
```

```
        for (int i = 0; i < key.length(); i++) {
            decryptMap.put(key.charAt(i), alphabet.charAt(i));
        }
        StringBuilder originalText = new StringBuilder();
        for (char c : ciphertext.toString().toCharArray()) {
            originalText.append(decryptMap.getOrDefault(c, c));
        }
        System.out.println("Decrypted Text: " + originalText);
        sc.close();
    }
}
```

OUTPUT:

```
Enter plaintext: BKIT BHALKI
Ciphertext: yprg yszopr
Decrypted Text: bkit bhalki
```

3. Implement Poly alphabetic Cipher

```
public class VigenereCipher {
  public static String encrypt(String text, final String key) {
    StringBuilder result = new StringBuilder();
    text = text.toUpperCase();
    String keyUpper = key.toUpperCase();
    for (int i = 0, j = 0; i < text.length(); i++) {
      char c = text.charAt(i);
      if (c < 'A' || c > 'Z') {
        result.append(c);
        continue;
      }
      result.append((char) ((c + keyUpper.charAt(j) - 2 * 'A') % 26 + 'A'));
      j = ++j % keyUpper.length();
    }
    return result.toString();
  }
  public static String decrypt(String text, final String key) {
    StringBuilder result = new StringBuilder();
    text = text.toUpperCase();
    String keyUpper = key.toUpperCase();
    for (int i = 0, j = 0; i < text.length(); i++) {
      char c = text.charAt(i);
      if (c < 'A' || c > 'Z') {
        result.append(c);
```

```
            continue;
        }
        result.append((char) ((c - keyUpper.charAt(j) + 26) % 26 + 'A'));
        j = ++j % keyUpper.length();
    }
    return result.toString();
  }
  public static void main(String[] args) {
    String key = "VIGENERECIPHER";
    String message = "Beware the Jabberwock, my son! The jaws that bite, the claws that
catch!";
    String encryptedMsg = encrypt(message, key);
    System.out.println("Original String: " + message);
    System.out.println("Encrypted message: " + encryptedMsg);
    String decryptedMsg = decrypt(encryptedMsg, key);
    System.out.println("Decrypted message: " + decryptedMsg);
  }
}
```

OUTPUT:

```
Original String:
Beware the Jabberwock, my son! The jaws that bite, the claws that catch!
Encrypted message:
WMCEEI KLG RPIFVMEUGX, QP WQV! IOI AVEY XUEK FKBT, ALV XTGAF XYEV KPAGY!
Decrypted message:
BEWARE THE JABBERWOCK, MY SON! THE JAWS THAT BITE, THE CLAWS THAT CATCH!
```

4. Encrypt the given text using Play fair Cipher with the given key. Also perform the decryption operation

```
import java.util.*;

public class PlayfairCipher {

   private char[][] matrix = new char[5][5];

   public PlayfairCipher(String key) {

      generateMatrix(key);

   }

   private void generateMatrix(String key) {

      String alphabet = "ABCDEFGHIKLMNOPQRSTUVWXYZ"; // 'J' is excluded

      String combined = (key.toUpperCase().replace("J", "I") + alphabet);

      StringBuilder uniqueStr = new StringBuilder();

      for (char c : combined.toCharArray()) {

         if (uniqueStr.indexOf(String.valueOf(c)) == -1) uniqueStr.append(c);

      }

      for (int i = 0; i < 25; i++) {

         matrix[i / 5][i % 5] = uniqueStr.charAt(i);

      }

   }

   private String formatText(String text, boolean encrypt) {

      text = text.toUpperCase().replace("J", "I").replaceAll("[^A-Z]", "");

      if (!encrypt) return text; // Ciphertext should already be even

      StringBuilder sb = new StringBuilder(text);

      for (int i = 0; i < sb.length() - 1; i += 2) {

         if (sb.charAt(i) == sb.charAt(i + 1)) sb.insert(i + 1, 'X');
```

```
    }
    if (sb.length() % 2 != 0) sb.append('X');
    return sb.toString();
  }
  private int[] findPos(char c) {
    for (int r = 0; r < 5; r++) {
      for (int col = 0; col < 5; col++) {
        if (matrix[r][col] == c) return new int[]{r, col};
      }
    }
    return null;
  }
  public String process(String text, boolean encrypt) {
    String formatted = formatText(text, encrypt);
    StringBuilder result = new StringBuilder();
    int delta = encrypt ? 1 : 4; // Use +1 for encryption, -1 (mod 5 is +4) for decryption
    for (int i = 0; i < formatted.length(); i += 2) {
      int[] a = findPos(formatted.charAt(i));
      int[] b = findPos(formatted.charAt(i + 1));
      if (a[0] == b[0]) { // Same Row
        result.append(matrix[a[0]][(a[1] + delta) % 5]);
        result.append(matrix[b[0]][(b[1] + delta) % 5]);
      } else if (a[1] == b[1]) { // Same Column
        result.append(matrix[(a[0] + delta) % 5][a[1]]);
        result.append(matrix[(b[0] + delta) % 5][b[1]]);
```

```
        } else { // Rectangle Rule
            result.append(matrix[a[0]][b[1]]);
            result.append(matrix[b[0]][a[1]]);
        }
    }
    return result.toString();
  }
  public static void main(String[] args) {
    PlayfairCipher pc = new PlayfairCipher("MONARCHY");
    String original = "INSTRUMENTS";
    String encrypted = pc.process(original, true);
    String decrypted = pc.process(encrypted, false);
    System.out.println("Key: MONARCHY");
    System.out.println("Plaintext:  " + original);
    System.out.println("Encrypted:  " + encrypted);
    System.out.println("Decrypted:  " + decrypted);
  }
}
```

OUTPUT:

```
Key: MONARCHY
Plaintext:  INSTRUMENTS
Encrypted:  GATLMZCLRQXA
Decrypted:  INSTRUMENTSX
```

5. Implement Encryption and Decryption techniques in Hill Cipher

```
import java.util.Scanner;
public class HillCipher {
  // Modular multiplicative inverse of determinant modulo 26
  private static int modInverse(int n) {
    n %= 26;
    for (int x = 1; x < 26; x++) {
      if ((n * x) % 26 == 1) return x;
    }
    return -1;
  }
  public static String encrypt(String text, int[][] key) {
    StringBuilder res = new StringBuilder();
    for (int i = 0; i < text.length(); i += 2) {
      int p1 = text.charAt(i) - 'A';
      int p2 = text.charAt(i + 1) - 'A';
      res.append((char) (((key[0][0] * p1 + key[0][1] * p2) % 26) + 'A'));
      res.append((char) (((key[1][0] * p1 + key[1][1] * p2) % 26) + 'A'));
    }
    return res.toString();
  }
  public static String decrypt(String cipher, int[][] key) {
    int det = (key[0][0] * key[1][1] - key[0][1] * key[1][0]) % 26;
    if (det < 0) det += 26;
    int invDet = modInverse(det);
```

```
        // Adjugate matrix modulo 26
        int[][] invKey = {
            { (key[1][1] * invDet) % 26, ((-key[0][1] * invDet) % 26 + 26) % 26 },
            { ((-key[1][0] * invDet) % 26 + 26) % 26, (key[0][0] * invDet) % 26 }
        };
        return encrypt(cipher, invKey); // Decryption uses same logic with inverse key
    }
    public static void main(String[] args) {
        int[][] key = {{3, 3}, {2, 5}}; // Valid key: det=9, coprime to 26
        String msg = "HELP";
        String encrypted = encrypt(msg, key);
        String decrypted = decrypt(encrypted, key);
        System.out.println("Original: " + msg);
        System.out.println("Encrypted: " + encrypted);
        System.out.println("Decrypted: " + decrypted);
    }
}
```

OUTPUT:

```
Original: HELP
Encrypted: HIAT
Decrypted: HELP
```

6. Demonstrate Single and Double Transposition techniques

```
import java.util.*;
public class Transposition {
   // Helper to get column order based on key (e.g., "COL" -> {0, 2, 1})
   private static Integer[] getColumnOrder(String key) {
      int n = key.length();
      Integer[] order = new Integer[n];
      for (int i = 0; i < n; i++) order[i] = i;
      Arrays.sort(order, Comparator.comparingInt(i -> key.charAt(i)));
      return order;
   }
   public static String encrypt(String text, String key) {
      int col = key.length();
      int row = (int) Math.ceil((double) text.length() / col);
      char[][] grid = new char[row][col];
      // Fill row-wise
      int k = 0;
      for (int i = 0; i < row; i++) {
         for (int j = 0; j < col; j++) {
            grid[i][j] = (k < text.length()) ? text.charAt(k++) : 'X'; // Padding
         }
      }
      // Read column-wise based on key order
      StringBuilder result = new StringBuilder();
      for (int c : getColumnOrder(key)) {
```

```
            for (int r = 0; r < row; r++) result.append(grid[r][c]);
        }
        return result.toString();
    }
    public static void main(String[] args) {
        String msg = "HELLOWORLD";
        String key1 = "BCA";
        String key2 = "CAB";
        // Single Transposition
        String singleEnc = encrypt(msg, key1);
        // Double Transposition (Pass 1 result through Pass 2)
        String doubleEnc = encrypt(singleEnc, key2);
        System.out.println("Original: " + msg);
        System.out.println("Single Transposition: " + singleEnc);
        System.out.println("Double Transposition: " + doubleEnc);
    }
}
```

OUTPUT:

```
Original: HELLOWORLD
Single Transposition: LWLXHLODEORX
Double Transposition: WHDRLLEXLXOO
```

7. Implement Simple DES/DES Algorithm

```
import javax.crypto.Cipher;

import javax.crypto.KeyGenerator;

import javax.crypto.SecretKey;

import javax.crypto.spec.SecretKeySpec;

import java.util.Base64;

public class SimpleDES {

   private static SecretKey secretKey;

   private static Cipher cipher;

   private static final String ALGORITHM = "DES";

   private static final String TRANSFORMATION = "DES/ECB/PKCS5Padding"; // ECB mode,
standard padding

   public static void main(String[] args) throws Exception {

      // 1. Generate the key (or use a predefined one)

      generateKey();

      String originalMessage = "Hello, World! This is a secret message.";

      System.out.println("Original Message: " + originalMessage);

      // 2. Encrypt the message

      String encryptedMessage = encrypt(originalMessage);

      System.out.println("Encrypted Message (Base64): " + encryptedMessage);

      // 3. Decrypt the message

      String decryptedMessage = decrypt(encryptedMessage);

      System.out.println("Decrypted Message: " + decryptedMessage);

   }

   /**
```

```
 * Generates a new DES secret key.
 */
public static void generateKey() throws Exception {
    KeyGenerator keyGen = KeyGenerator.getInstance(ALGORITHM);
    secretKey = keyGen.generateKey();
    cipher = Cipher.getInstance(TRANSFORMATION);
}
/**
 * Optional: Recreate key from existing byte array
 */
public static void useKey(byte[] keyBytes) throws Exception {
    secretKey = new SecretKeySpec(keyBytes, ALGORITHM);
    cipher = Cipher.getInstance(TRANSFORMATION);
}
/**
 * Encrypts a plaintext string using the generated key.
 */
public static String encrypt(String plaintext) throws Exception {
    cipher.init(Cipher.ENCRYPT_MODE, secretKey);
    byte[] encryptedBytes = cipher.doFinal(plaintext.getBytes());
    // Use Base64 to convert byte array to a string for easy printing/storage
    return Base64.getEncoder().encodeToString(encryptedBytes);
}
/**
 * Decrypts an encrypted string (Base64 encoded) back to plaintext.
```

```
    */
    public static String decrypt(String encryptedText) throws Exception {
        cipher.init(Cipher.DECRYPT_MODE, secretKey);
        // Decode Base64 string back to byte array
        byte[] decodedBytes = Base64.getDecoder().decode(encryptedText);
        byte[] decryptedBytes = cipher.doFinal(decodedBytes);
        return new String(decryptedBytes);
    }
}
```

OUTPUT:

```
Original: HELLOWORLD
Single Transposition: LWLXHLODEORX
Double Transposition: WHDRLLEXLXOO
```

8. Generate Pseudo random numbers using Linear Congruential method

```
public class LinearCongruentialGenerator {

   private long seed;

   private final long multiplier;

   private final long increment;

   private final long modulus;

   // Constructor with a custom seed

   public LinearCongruentialGenerator(long seed, long multiplier, long increment, long
modulus) {

      this.seed = seed;

      this.multiplier = multiplier;

      this.increment = increment;

      this.modulus = modulus;

   }

   // Constructor using system time as default seed

   public LinearCongruentialGenerator() {

      this(System.currentTimeMillis(), 1664525L, 1013904223L, 4294967296L); //
Parameters from Numerical Recipes

   }

   /**

    * Generates the next pseudo-random long integer in the sequence.

    * The result is non-negative and less than the modulus.

    * @return The next pseudo-random long.

    */

   public long nextLong() {
```

```
        seed = (multiplier * seed + increment) % modulus;
        // Ensure the result is non-negative
        return seed < 0 ? seed + modulus : seed;
    }
    /**
     * Generates the next pseudo-random integer between 0 (inclusive) and bound
(exclusive).
     * @param bound The upper bound (exclusive).
     * @return The next pseudo-random integer.
     */
    public int nextInt(int bound) {
        // Generate next long and scale it to the desired bound
        long nextVal = nextLong();
        return (int) (nextVal % bound);
    }
    /**
     * Generates the next pseudo-random double between 0.0 (inclusive) and 1.0 (exclusive).
     * @return The next pseudo-random double.
     */
    public double nextDouble() {
        // Generate next long and scale it to the range [0.0, 1.0)
        return (double) nextLong() / (double) modulus;
    }
    public static void main(String[] args) {
        // Example usage:
```

```
        LinearCongruentialGenerator lcg = new LinearCongruentialGenerator();
        System.out.println("Generating 5 pseudo-random numbers:");
        for (int i = 0; i < 5; i++) {
            System.out.println(lcg.nextLong());
        }
        System.out.println("\nGenerating 5 pseudo-random integers between 0 and 100:");
        for (int i = 0; i < 5; i++) {
            System.out.println(lcg.nextInt(100));
        }
        System.out.println("\nGenerating 5 pseudo-random doubles between 0.0 and 1.0:");
        for (int i = 0; i < 5; i++) {
            System.out.println(lcg.nextDouble());
        }
    }
}
```

OUTPUT:

```
Generating 5 pseudo-random numbers:
3591983216
213166351
1601078050
3952945177
3151295140

Generating 5 pseudo-random integers between 0 and 100:
91
98
81
0
3

Generating 5 pseudo-random doubles between 0.0 and 1.0:
0.1964458846487105
0.3222128676716238
0.609629082493484
0.08460543933324516
0.10497414274141192
```

9. Generate Pseudo random numbers using Blum Blum Shub Generator

```
import java.math.BigInteger;

import java.security.SecureRandom;

public class BlumBlumShub {

   private BigInteger n;

   private BigInteger x;

   /**

    * Initializes the Blum Blum Shub generator.

    * @param p A large prime number such that p % 4 == 3.

    * @param q A large prime number such that q % 4 == 3 and p != q.

    * @param seed A random integer co-prime to n (gcd(seed, n) == 1).

    */

   public BlumBlumShub(BigInteger p, BigInteger q, BigInteger seed) {

      if (!p.isProbablePrime(100) || p.remainder(BigInteger.valueOf(4)).intValue() != 3) {

         throw new IllegalArgumentException("p must be a Blum prime (p % 4 == 3)");

      }

      if (!q.isProbablePrime(100) || q.remainder(BigInteger.valueOf(4)).intValue() != 3) {

         throw new IllegalArgumentException("q must be a Blum prime (q % 4 == 3)");

      }

      if (p.equals(q)) {

         throw new IllegalArgumentException("p and q must be distinct");

      }

      this.n = p.multiply(q);

      // Ensure seed is co-prime to n
```

```
    if (!seed.gcd(n).equals(BigInteger.ONE)) {
        throw new IllegalArgumentException("Seed must be co-prime to n");
    }
    // Initialize the state x0 = seed^2 mod n
    this.x = seed.modPow(BigInteger.TWO, this.n);
}
/**
 * Generates the next pseudo-random bit (least significant bit).
 * @return The next pseudo-random bit (0 or 1).
 */
public int generateNextBit() {
    // Calculate the next state: x_i+1 = x_i^2 mod n
    this.x = this.x.modPow(BigInteger.TWO, this.n);
    // Extract the least significant bit (x_i+1 mod 2)
    return this.x.remainder(BigInteger.TWO).intValue();
}
// Example Usage
public static void main(String[] args) {
    // Choose large Blum primes p and q (for a real application, use much larger primes)
    // Ensure they satisfy the condition p % 4 == 3
    BigInteger p = new BigInteger("30000000091"); // Example large prime
    BigInteger q = new BigInteger("40000000003"); // Example large prime
    // Choose a seed co-prime to n
    BigInteger seed = new BigInteger("4882516701");
    BlumBlumShub bbs = new BlumBlumShub(p, q, seed);
```

```
        System.out.println("Generating 20 pseudo-random bits:");

        StringBuilder bitSequence = new StringBuilder();

        for (int i = 0; i < 20; i++) {

            bitSequence.append(bbs.generateNextBit());

        }

        System.out.println(bitSequence.toString());

    }

}
```

OUTPUT:

```
Generating 20 pseudo-random bits:
00010000100001000110
```

10. Implement RSA Algorithm

```
import javax.crypto.Cipher;
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.PrivateKey;
import java.security.PublicKey;
import java.util.Base64;
public class RSAExample {
   public static void main(String[] args) throws Exception {
      // 1. Generate RSA Key Pair (2048 bits is standard)
      KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");
      keyGen.initialize(2048);
      KeyPair pair = keyGen.generateKeyPair();
      PublicKey publicKey = pair.getPublic();
      PrivateKey privateKey = pair.getPrivate();
      String originalMessage = "Hello, this is a secret!";
      // 2. Encryption
      Cipher encryptCipher = Cipher.getInstance("RSA/ECB/PKCS1Padding");
      encryptCipher.init(Cipher.ENCRYPT_MODE, publicKey);
      byte[] encryptedBytes = encryptCipher.doFinal(originalMessage.getBytes());
      String encodedMessage = Base64.getEncoder().encodeToString(encryptedBytes);
      // 3. Decryption
      Cipher decryptCipher = Cipher.getInstance("RSA/ECB/PKCS1Padding");
      decryptCipher.init(Cipher.DECRYPT_MODE, privateKey);
```

```
    byte[] decryptedBytes =
decryptCipher.doFinal(Base64.getDecoder().decode(encodedMessage));

    String decryptedMessage = new String(decryptedBytes);

    System.out.println("Original: " + originalMessage);

    System.out.println("Encrypted (Base64): " + encodedMessage);

    System.out.println("Decrypted: " + decryptedMessage);

  }

}
```

OUTPUT:

```
Original: Hello, this is a secret!
Encrypted (Base64):
dLpiIyiagpcJ4TWu7/cfdJhaUEGMhcOjjF/4H140QJo33EDnhtdIhmN4xzHzDXob33FVR/sGbMMB4dxev0h5i
CSzXp/Nb8QsJHeT59aqFNAj77dDR4HrIDVs4fEcdFhy9Tql2HQRgJm+++YXxFyZETt1MwGS3ju1vZsu6xTT5B
5iYuMKdCmF2NOyXES/kQdMHUOlMyJsYCbEu3zsh77hnTx2E/e8RUwfzWi+vGc0AWHlKF0uMfC3jG3aeaG6cCc
j+ch57FvAS/K2HADT9DOjg+ZmyVDjFdoZY+q5bPz/QaciNAa5dxnJTilrvV+ZLaEhn7kV3OENuCfsEuFn6Vgi
MQ==
Decrypted: Hello, this is a secret!
```

11. Demonstrate Diffie Hellman Key exchange Algorithm

```
import java.math.BigInteger;

import java.util.Scanner;

public class DiffieHellmanDemo {

   public static void main(String[] args) {

      Scanner stdin = new Scanner(System.in);

      // 1. Publicly agreed upon parameters (g and p)

      // For demonstration, we use small values. In real applications, these are large primes.

      BigInteger p = new BigInteger("23"); // Prime modulus

      BigInteger g = new BigInteger("5");  // Generator/base

      System.out.println("Publicly agreed Prime (p): " + p);

      System.out.println("Publicly agreed Base (g): " + g);

      // 2. Alice and Bob choose their private keys (secret integers)

      System.out.println("\nEnter Alice's private key (a): ");

      BigInteger a = new BigInteger(stdin.next());

      System.out.println("Enter Bob's private key (b): ");

      BigInteger b = new BigInteger(stdin.next());

      // 3. Alice and Bob calculate and exchange their public keys

      // Alice's public key A = g^a mod p

      BigInteger A = g.modPow(a, p);

      System.out.println("\nAlice's public value (A) sent to Bob: " + A);

      // Bob's public key B = g^b mod p

      BigInteger B = g.modPow(b, p);

      System.out.println("Bob's public value (B) sent to Alice: " + B);

      // 4. Alice and Bob compute the shared secret key independently
```

```
        // Alice calculates s = B^a mod p (using Bob's public key and her private key)
        BigInteger secretKeyAlice = B.modPow(a, p);
        System.out.println("\nSecret Key computed by Alice: " + secretKeyAlice);
        // Bob calculates s = A^b mod p (using Alice's public key and his private key)
        BigInteger secretKeyBob = A.modPow(b, p);
        System.out.println("Secret Key computed by Bob: " + secretKeyBob);
        // Verification
        if (secretKeyAlice.equals(secretKeyBob)) {
            System.out.println("\nKey exchange successful! Shared secret is: " + secretKeyAlice);
        } else {
            System.out.println("\nError: Keys do not match.");
        }
        stdin.close();
    }
}
```

OUTPUT:

```
Publicly agreed Prime (p): 23
Publicly agreed Base (g): 5

Enter Alice's private key (a):
15
Enter Bob's private key (b):
20

Alice's public value (A) sent to Bob: 19
Bob's public value (B) sent to Alice: 12

Secret Key computed by Alice: 13
Secret Key computed by Bob: 13

Key exchange successful! Shared secret is: 13
```

12. Implement Fermat's and Euler's Theorem

```
import java.util.Scanner;

import java.math.BigInteger;

public class NumberTheoryTheorems {

    /**

     * Calculates (a^b) % m using modular exponentiation.

     * This is an efficient way to handle large exponents.

     */

    public static long power(long base, long exp, long mod) {

        long res = 1;

        base %= mod;

        while (exp > 0) {

            if ((exp % 2) == 1) {

                res = (res * base) % mod;

            }

            base = (base * base) % mod;

            exp /= 2;

        }

        return res;

    }

    /**

     * Implements Fermat's Little Theorem:

     * If p is a prime number and a is an integer not divisible by p,

     * then a^(p-1) ≡ 1 (mod p).

     *
```

```
 * This function checks the condition for given a and a prime p.
 */
public static boolean checkFermatsLittleTheorem(long a, long p) {
    // Condition: p must be prime and not divide a (or gcd(a, p) == 1)
    if (!isPrime(p) || (a % p == 0)) {
        System.out.println("Conditions for Fermat's Little Theorem not met.");
        return false;
    }
    // Calculate a^(p-1) mod p
    long result = power(a, p - 1, p);
    System.out.println(a + "^(" + p + "-1) mod " + p + " = " + result);
    return result == 1;
}
/**
 * Calculates Euler's Totient function phi(n).
 * This counts the number of positive integers less than or equal to n
 * that are relatively prime to n (gcd(a, n) == 1).
 */
public static long phi(long n) {
    long result = n;
    for (long i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            while (n % i == 0) {
                n /= i;
            }
```

```
            result -= result / i;
        }
    }
    if (n > 1) {
        result -= result / n;
    }
    return result;
}
/**
 * Implements Euler's Theorem (Euler-Fermat Theorem):
 * If a and n are coprime positive integers (gcd(a, n) == 1),
 * then a^phi(n) ≡ 1 (mod n), where phi(n) is Euler's totient function.
 */
public static boolean checkEulersTheorem(long a, long n) {
    // Condition: a and n must be coprime (gcd(a, n) == 1)
    if (gcd(a, n) != 1) {
        System.out.println("Conditions for Euler's Theorem not met (a and n must be
coprime).");
        return false;
    }
    // Calculate a^phi(n) mod n
    long phi_n = phi(n);
    long result = power(a, phi_n, n);
    System.out.println(a + "^phi(" + n + ") mod " + n + " = " + result + " (phi(" + n + ") = " +
phi_n + ")");
```

```
    return result == 1;
  }
  /**
   * Helper function to calculate the Greatest Common Divisor (GCD)
   * using the Euclidean algorithm.
   */
  public static long gcd(long a, long b) {
    while (b != 0) {
      long temp = b;
      b = a % b;
      a = temp;
    }
    return a;
  }
  /**
   * A simple primality test (for small numbers)
   */
  public static boolean isPrime(long n) {
    if (n <= 1) return false;
    for (long i = 2; i * i <= n; i++) {
      if (n % i == 0) return false;
    }
    return true;
  }
  public static void main(String[] args) {
```

```
    System.out.println("--- Fermat's Little Theorem Example ---");
    long a1 = 3;
    long p1 = 5; // Prime number
    System.out.println("Checking a = " + a1 + ", p = " + p1);
    boolean fermatResult = checkFermatsLittleTheorem(a1, p1);
    System.out.println("Fermat's Little Theorem holds: " + fermatResult);

    System.out.println("\n--- Euler's Theorem Example ---");
    long a2 = 10;
    long n2 = 9; // Not a prime, but coprime to 10
    System.out.println("Checking a = " + a2 + ", n = " + n2);
    boolean eulerResult = checkEulersTheorem(a2, n2);
    System.out.println("Euler's Theorem holds: " + eulerResult);
    System.out.println("\n--- Euler's Theorem with a prime n (special case) ---");
    long a3 = 4;
    long n3 = 5; // Prime number
    System.out.println("Checking a = " + a3 + ", n = " + n3);
    boolean eulerPrimeResult = checkEulersTheorem(a3, n3);
    System.out.println("Euler's Theorem holds: " + eulerPrimeResult + " (demonstrating it
generalizes Fermat's)");
  }
}
```

OUTPUT:

```
--- Fermat's Little Theorem Example ---
Checking a = 3, p = 5
3^(5-1) mod 5 = 1
Fermat's Little Theorem holds: true

--- Euler's Theorem Example ---
Checking a = 10, n = 9
10^phi(9) mod 9 = 1 (phi(9) = 6)
Euler's Theorem holds: true

--- Euler's Theorem with a prime n (special case) ---
Checking a = 4, n = 5
4^phi(5) mod 5 = 1 (phi(5) = 4)
Euler's Theorem holds: true (demonstrating it generalizes Fermat's)
```